

ECDAT (SIH26164) - 36-Hour Execution Plan

Core Principle: Ruthless Prioritization

Vision: Multi-modal cryptographic intelligence platform.

MVP: Source + dependency + certificate discovery -> CBOM -> Mosca risk -> PQC recommendations -> dashboard.

Stretch (only if time): Container scanning, binary NTT detection, blast-radius graph.

Skip: Cloud KMS, HSM, runtime telemetry, CEL policy engine, agentic layer.

Pre-Hackathon Preparation (Do This Before Day 1)

1. Environment & Repo Setup

- GitHub repo with main and dev branches.
- Docker Compose with services: backend (FastAPI), frontend (React), scanner (Python).
- Pre-install dependencies: tree-sitter, semgrep, cryptography, pyasn1, cyclonedx-python-lib, fastapi, uvicorn, react, vis-network, recharts.

2. Test Corpus (Critical)

Create a synthetic repository with known crypto assets:

- Python: RSA.generate(2048), ECDSA, AES-GCM, ssl.SSLContext.
- Java: BouncyCastle usage, KeyPairGenerator.getInstance("RSA").
- JavaScript: crypto.createSign('RSA-SHA256'), tls.createSecureContext.
- Certificates: self-signed RSA-2048, ECDSA P-256.
- Dependencies: cryptography==44.0, openssl, bouncycastle.
- Ground truth JSON listing every expected finding with file, line, algorithm, key size.

3. CBOM Schema Stub

Create a JSON template mapping CycloneDX crypto-asset fields:

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.6",
  "components": [
    {
      "type": "cryptographic-asset",
      "name": "RSA-2048",
      "cryptoProperties": {
        "assetType": "algorithm",
        "algorithmProperties": {
          "primitive": "signature",
          "parameterSetIdentifier": "2048",
          "classicalSecurityLevel": 112,
          "nistQuantumSecurityLevel": 0
        }
      },
      "evidence": {
        "occurrences": [
          { "location": "auth/signing.py:184", "line": 184, "confidence": 0.97 }
        ]
      }
    }
  ]
}
```

```

    }
  }
]
}

```

4. Risk Model Constants

- GRI Q-day distribution: use midline 2035, uncertainty ± 5 years. Document assumption.
- Mosca parameters: X (data lifetime) default 10 years, Y (migration time) default 6 years.
- Weights for Layer 2: criticality 30%, exposure 25%, blast radius 20%, agility 15%, data sensitivity 10%.

Hackathon Timeline (36 Hours)

Hour 0-4: Core Scanner Engine

Goal: Working AST scanner for Python + JavaScript + Java.

- Lead: Backend dev.
- Use tree-sitter with language grammars.
- Write Semgrep rules for:
 - RSA, ECDSA, ECDH, DH, AES, SHA, MD5, DES.
 - Key sizes and modes (GCM, CBC).
 - Crypto library imports (cryptography, OpenSSL, BouncyCastle).
- Output: JSON findings with file, line, algorithm, key size, confidence, evidence snippet.

Deliverable: scanner.py that scans a repo and outputs findings.json.

Hour 4-8: Dependency & Certificate Scanners

Goal: Add dependency and X.509 analysis.

- Lead: Backend dev.
- Dependency scanner:
 - Parse requirements.txt, package.json, pom.xml, go.mod.
 - Map known crypto libs to algorithms (OpenSSL -> RSA, ECDSA; BouncyCastle -> AES, RSA).
- Certificate scanner:
 - Use cryptography library to parse PEM/DER.
 - Extract: subject, issuer, validity, key type, key size, signature algorithm.
- Merge into findings.json with source: "dependency" or source: "certificate".

Deliverable: Unified findings JSON with ~50 assets from test corpus.

Hour 8-12: CBOM Generator & Evidence Graph

Goal: Convert findings to CycloneDX CBOM and build asset graph.

- Lead: Backend dev + data modeler.
- Map findings to CycloneDX crypto-asset schema.
- Build a graph (NetworkX) with nodes: Application, Library, Algorithm, Certificate, Protocol.
- Edges: uses, implements, protects, dependsOn.
- Store as JSON: {nodes: [...], edges: [...]}.
 - Application: {name: "my-app", version: "1.0.0", type: "Application", hashes: [{"alg": "SHA-256", "value": "abc123"}]}
 - Library: {name: "my-lib", version: "1.0.0", type: "Library", hashes: [{"alg": "SHA-256", "value": "def456"}]}
 - Algorithm: {name: "my-alg", version: "1.0.0", type: "Algorithm", hashes: [{"alg": "SHA-256", "value": "ghi789"}]}
 - Certificate: {name: "my-cert", version: "1.0.0", type: "Certificate", hashes: [{"alg": "SHA-256", "value": "jkl012"}]}
 - Protocol: {name: "my-prot", version: "1.0.0", type: "Protocol", hashes: [{"alg": "SHA-256", "value": "mno345"}]}

Deliverable: cbom.json and graph.json from test corpus.

Hour 12-16: Quantum Risk Engine (Mosca Monte Carlo)

Goal: Implement Layer 1 risk scoring.

- Lead: ML/risk dev.
- Implement Monte Carlo simulation:

```
import numpy as np
N = 200000
Tq = np.random.normal(2035, 3, N) # Q-day distribution
X = 10 # data lifetime
Y = 6 # migration time
Ts = 2026
Tc = Ts + Y
f = np.minimum(1, np.maximum(0, (np.minimum(Tq, Tc) - (Tq - X)) / X))
P_f_gt_0 = np.mean(f > 0)
E_f = np.mean(f)
```

- Compute for each asset based on its type (KEM vs signature vs symmetric).
- Assign risk score: $\text{risk} = P_f_gt_0 \cdot 0.4 + E_f \cdot 0.3 + \text{criticality} \cdot 0.3$ (adjust weights).

Deliverable: risk_scores.json mapping each asset to risk level (Critical/High/Medium/Low).

Hour 16-20: PQC Recommendation Engine

Goal: Map vulnerable assets to PQC alternatives.

- Lead: Crypto researcher.
- Simple rule-based mapping:

Current	Primitive	Recommendation	Hybrid Option
RSA-2048	KEM	ML-KEM-768	X25519 + ML-KEM-768
RSA-2048	Signature	ML-DSA-65	RSA + ML-DSA-65
ECDSA P-256	Signature	ML-DSA-44	ECDSA + ML-DSA-44
ECDH	KEM	ML-KEM-768	X25519 + ML-KEM-768
AES-128	Symmetric	AES-256	—
SHA-256	Hash	SHA-384	—

- Add rationale and performance note (e.g., "ML-DSA signature size ~2.4KB vs ECDSA 64B").

Deliverable: recommendations.json per asset.

Hour 20-26: Frontend Dashboard

Goal: Interactive GUI for inventory, risk, and recommendations.

- Lead: Frontend dev.
- Pages:

1. Dashboard: Summary cards (total assets, critical count, risk distribution), heatmap.
2. Inventory: Table with filters (algorithm, risk, source). Click row -> detail panel.
3. Asset Detail: Evidence (file, line, snippet), risk breakdown (Mosca + context), recommendation.

4. Graph View: Interactive network graph (vis-network) showing dependencies.
5. CBOM Export: Download button for cbom.json.
- Use Recharts for risk distribution charts.

Deliverable: React app running on localhost:3000, consuming FastAPI endpoints.

Hour 26–30: Integration, Polish, and Demo Data

Goal: End-to-end flow working on test corpus.

- All hands.
- FastAPI endpoints:
 - POST /scan -> returns findings, CBOM, risk, recommendations.
 - GET /assets -> list assets.
 - GET /asset/{id} -> detail.
 - GET /graph -> graph JSON.
- Seed with a realistic demo repo (e.g., enterprise-payment-platform).
- Pre-run scan to populate dashboard.
- Record a backup demo video in case live demo fails.

Deliverable: Working demo: upload repo -> scan -> view dashboard -> export CBOM.

Hour 30–34: Stretch Goals (Only If Core Is Solid)

Pick one:

- Container scanner: Unpack OCI image, scan filesystem for certs and configs.
- Binary NTT detection: Search for ML-KEM ζ -tables in a stripped binary. (High risk, high reward.)
- Blast-radius graph: Highlight services affected by migrating one asset.

If none is feasible, use time to refine UI and add evidence traceability.

Hour 34–36: Final Prep

- Presentation: 10 slides max.
- Demo script: 3-minute walkthrough.
- Backup: Screenshots and video.
- Pitch: Emphasize evidence fusion + probabilistic Mosca + actionable migration.

Team Roles (Assuming 4–6 People)

Role	Responsibilities
Backend Lead	Scanner engine, AST, dependency, certificate parsing.
Risk/ML Dev	Monte Carlo, risk scoring, recommendation mapping.
Frontend Lead	Dashboard, graph view, asset detail, CBOM export.
Full-Stack/Integration	FastAPI endpoints, data flow, Docker, deployment.
Crypto Researcher	PQC mappings, standards references, validation.
Presenter/PM	Slides, demo script, time management, backup video.

Tech Stack (Final)

Layer	Technology
-------	------------

Scanning	Python, Tree-sitter, Semgrep, cryptography, pyasn1
Backend	FastAPI, Uvicorn, NetworkX, NumPy
Frontend	React, Recharts, vis-network, TailwindCSS
CBOM	CycloneDX Python library
Deployment	Docker Compose
Demo	Pre-seeded repo, localhost

Demo Script (3 Minutes)

1. Problem (20s): "Enterprises don't know where crypto lives. Quantum computers will break RSA/ECC. Migration is urgent."
2. Scan (30s): Upload enterprise-payment-platform repo. Show progress bar. "ECDAT scans source, dependencies, certificates."
3. Inventory (40s): Dashboard shows 146 assets, 11 critical. Click RSA-2048.
4. Asset Detail (40s): Show evidence: file auth/signing.py:184, AST call RSA.generate(2048), dependency OpenSSL. Show Mosca: $X+Y>Z$, $P(f>0)=0.73$, $E[f]=0.54$.
5. Recommendation (30s): "Migrate to ML-DSA-65 or hybrid RSA+ML-DSA. Here's the agility score and migration roadmap."
6. CBOM Export (10s): Download CycloneDX CBOM. "Standardized, auditable, actionable."
7. Close (10s): "ECDAT: from cryptographic blindness to quantum readiness."

Risk Mitigation

Risk	Mitigation
Scanner misses assets	Use test corpus with ground truth; measure F1.
Monte Carlo criticized as arbitrary	Document GRI source; show sensitivity analysis.
Frontend not ready	Use static JSON fallback; pre-render screenshots.
Binary NTT too hard	Drop it; focus on core.
Time overrun	Cut stretch goals at Hour 30.

Success Metrics for SIH

- Discovery: $F1 \geq 0.75$ on test corpus.
- CBOM: Valid CycloneDX v1.6 JSON.
- Risk: Monte Carlo outputs $P(f>0)$ and $E[f]$ with clear visualization.
- Recommendation: At least 5 correct PQC mappings.
- Demo: End-to-end scan -> dashboard -> export in under 2 minutes.

Final Pitch Line

"ECDAT fuses AST, dependency, and certificate evidence into a standardized CBOM, quantifies quantum risk via probabilistic Mosca simulation, and delivers an actionable PQC migration roadmap — turning cryptographic blindness into quantum readiness."

This plan gives you a realistic, high-impact MVP in 36 hours while preserving the core innovations from your blueprint. Execute ruthlessly, demo flawlessly.